

EPFL CIVIL-127, Lab 6

March 24, 2026

This week, you will finish implementing Marmotte Sweeper.

6.1

Create a subfolder `CIVIL-127/lab6`. Copy the partial Marmotte Sweeper code you have from `CIVIL-127/lab5/marmotte_sweeper` to `CIVIL-127/lab6/marmotte_sweeper`. Update your `launch.json` to launch the code in `lab6.marmotte_sweeper.main`. Fix the imports in your code (so that your `lab6/marmotte_sweeper/main.py` is importing `lab6.marmotte_sweeper.ui` and `lab6/marmotte_sweeper/ui.py` is importing `lab6.marmotte_sweeper.logic`)

- In `class MarmotteSweeper`
 - Implement a method `count`, which takes a position and returns the number of marmottes in the neighboring 8 cells
 - If you wrote unittests, add a test for `count()`
- In `class MarmotteSweeperUI`
 - Update `__init__` to load the remaining images from the `assets/` directory (`marmotte.png`, `0.png`, `1.png`, `2.png`, `3.png`, `4.png`, `5.png`, `6.png`, `7.png`, `8.png`).
 - Update the mouse button event handling code to call `flag` or `reveal` depending on which mouse button was pressed.
 - Update the draw method to draw different images based on the 4 cases that `cell()` can return: `UNREVEALED`, `MARMOTTE`, `REVEALED`, `FLAGGED`. If the cell is `REVEALED`, the draw method should call `count()` to decide which image to show.

Congrats, you should now have a functional game!

6.2 (optional)

Update `MarmotteSweeper`. When revealing a cell with 0 neighboring marmottes, all the neighbors are automatically revealed. And if the newly revealed cells have no neighboring marmottes, their neighbors are also revealed, and so on. You can implement this logic recursively or store which cells need to be revealed in a queue/stack/set and keep processing until the data structure is empty.

- In `class MarmotteSweeper`
 - Update the `reveal` method.
 - If you wrote tests, update your tests.

6.3 (optional)

Think about some improvements or twists you can make to the game and try to implement them. Show your variations to your friends and family!